Doc Technique

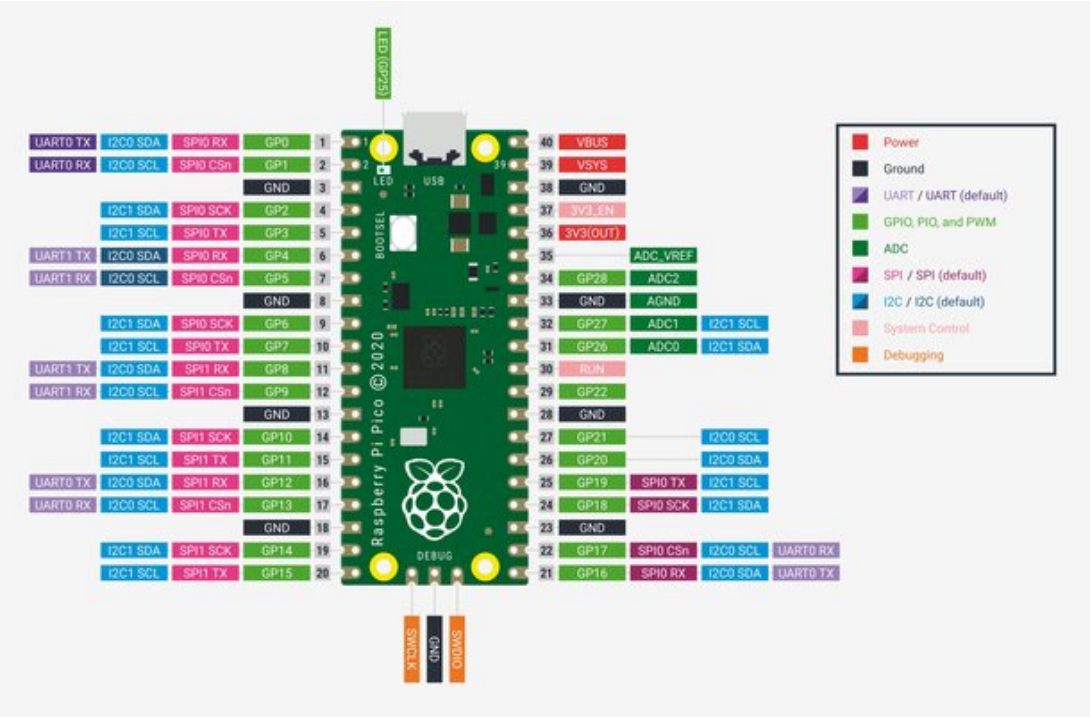
PicoGo Waveshare

1 - Liste des Composants

https://www.waveshare.com/wiki/PicoGo

- Standard Raspberry Pi Pico header, supports Raspberry Pi Pico series.
- Battery protection circuit: overcharge/discharge protection, over current protection, short circuit protection, reverse proof, more stable and safe operating.
- Recharge/Discharge circuit, allows programming/debugging concurrently while recharging.
- 5-ch infrared sensor, analog output, combined with PID algorithm, stable line tracking.
- Onboard multiple smart robot sensors like line tracking, obstacle avoidance, and no more messy wiring.
- 1.14inch IPS colorful LCD display, 240 x 135 pixels, 65K colors.
- Integrates Bluetooth module allows teleoperations like robot movement, RGB LED display color, buzzer, etc. by using mobile phone APP.
- N20 micro gear motors, with metal gears, low noise, and high accuracy.
- Colorful RGB LED, pretty cool!

2 - Schéma de montage



Module	GPIO Nb	Module	GPIO Nb
	GPIO#0	----	----
	GPIO#1	----	----
DSR Detection IR	GPIO#2	----	----
DSL Detection IR	GPIO#3	TRSensor ?	GPIO#28
Buzzer	GPIO#4	TRSensor ?	GPIO#27
Telecommande IR	GPIO#5	ADC0 Battery	GPIO#26
TRSensor ?	GPIO#6	----	----
TRSensor ?	GPIO#7	----	----
SPI1-DC OLED ST7789	GPIO#8	----	----
SPI1-CS OLED ST7789	GPIO#9	Neopixel 4 Leds	GPIO#22
SPI1-SCK OLED ST7789	GPIO#10	MotorB PWM	GPIO#21
SPI1-MOSI OLED ST7789	GPIO#11	MotorB IN2	GPIO#20
SPI1-RST OLED ST7789	GPIO#12	MotorB IN1	GPIO#19
SPI1-BL OLED ST7789	GPIO#13	MotorA IN1	GPIO#18
Ultrason HC-SR04 Trigger	GPIO#14	MotorA IN2	GPIO#17
Ultrason HC-SR04 Echo	GPIO#15	MotorA PWM	GPIO#16

BLE sur UART0 ???

3 - Code Micropython

.\Pico_Go\PicoGo_Code V2

Battery_Voltage.py	• Open the Battery_Voltage.py in Thonny IDE and run it. • LCD will display chip temperature, battery voltage, and power percentage. The percentage of electric quantity is obtained by simple linear conversion of voltage. The actual battery voltage and electric quantity are not linear, so there will be some errors in this percentage.
Bluetooth.py	• Open the bluetooth.py in the Thonny IDE, rename it as main.py, and save it to Raspberry Pi Pico. • Install PicoGo APP in your phone (only supports Android).
Infrared_obstacle_Avoidance.py	• Open the Infrared-Obstacle-Avoidance.py in Thonny IDE, rename it as main.py, and save it to Pico. Disconnect the USB cable and run it. • When there is no obstacle in front of the car, the green LED light in front of the car will be off. When the car meets an obstacle, the green LED light in front will be on. • If the LED light is not bright or keeps brightening, you can adjust two potentiometers on the bottom of the PicoGo, so that the LED is just out of state. The detection distance is the farthest. • Procedure phenomenon is no obstacle when the car straight, encountered obstacles when the car turns right.
IRremote.py	• Open the IRremote.py in ThonnyIDE and run it. • Press the Infrared controller to control the PicoGo. • 2, 8, 4, 6, 5 are used for forwarding, backward, turn left, turn right, and stop. You can press the - or + keys to adjust the speed and press EQ to restore the setting. • Different infrared remote controllers may have different key codes, if you use other controllers, you may need to modify the codes.
Line-Tracking.py	• Open the Line-Tracking.py file in Thonny IDE, rename it as main.py, and save it to Raspberry Pi Pico. • Tracking sensor can detect black lines with a white background (or white lines with a black background, need to modify program). • The tracking board can be made by sticking black tape in the white KT board. The width of the black track is 15mm. If the background color is too dark, the tracking effect will be affected. • After disconnecting the USB cable, running the demo, and putting the car in the black line, the car will rotate left and right, this is the car calibration stage. If the calibration phase operation error will directly affect the tracking effect.
Line-Tracking2.py	• Open a Line-Tracking2.py in Thonny IDE, rename it as main.py, and save it to Raspberry Pi Pico. • After disconnecting the USB cable, running the program, and putting the car in the black line, the car will rotate left and right to calibrate. After calibration, the black line will be run. • When there is an obstacle in front of the car, the car will stop and the buzzer will sound. After the obstacle is cleared, the car will continue to

	run. Pick up the car and the motor will stop. • During the calibration stage of the car, the four RGBS display red, green, and blue respectively. Change. The RGB LED will display the color light effect when tracking is running.
Motor.py	• The PicoGo will move forward, then backward, turn left, and turn right after running the codes.
ST7789.py	• Open the ST7789.py in Thonny IDE and run it. • After the program runs normally, LCD will display the string.
TRSensor.py	• Open the TRsensor.py in Thonny IDE and run it. • The shell interface will display the values of the five tracking sensors. The data range is 600~900 when the PicoGo is put on the white paper, and the data range is 0~50 when the PicoGo is put in the air.
Ultrasionc-Infrared-follow.py	• Open the Ultrasionc-Infrared-follow.py in Thonny, rename it as main.py, and save it to Raspberry Pi Pico. • Run the program after disconnecting the USB cable, place the object in the sensor of the car, and the car will automatically follow the object to move. • The following distance of the car can be set, the default following distance is 5cm, the car will stop when it is 5cm away from the object, and the car will continue to run when it is larger than 5cm and smaller than 7cm. • Turn left and right by infrared. • When the car is running, the RGB LED will display the color light effect.
Ultrasionc-Infrared-Obstacle-Avoidance.py	• Open the Ultrasionc-Infrared-Obstacle-Avoidance.py in Thonny, rename it as main.py, and save it to Raspberry Pi Pico. • Run the program after disconnecting the USB cable. Go straight when there is no obstacle, and turn right when there is an obstacle. The combination of ultrasonic and infrared has a better obstacle avoidance effect and a higher success rate.
Ultrasonic_Obstacle_Avoidance.py	• Open the Ultrasonic-Obstacle-Avoidance.py in Thonny IDE, rename it as main.py, and save it to Raspberry Pi Pico. • Run the program after disconnecting the USB cable. Go straight when there is no obstacle, and turn right when there is an obstacle.
Ultrasonic_Ranging.py	• Open the Ultrasionc_Ranging.py in Thonny IDE, the detected distance will be shown on the shell.
ws2812.py	• Open the WS2812.py in Thonny and run it. • Four colored LED lights at the bottom of the car will show red, yellow, green, cyan, blue, purple, and white, and then show the color light effect.

4 - Photos